

CAN AI Diagnostic Backend - Professional Upgrade Report

1) System Architecture

Pipeline: Multi-format CAN Parser -> Feature Engineering -> StandardScaler -> IsolationForest anomaly detection -> KMeans clustering -> anomaly intelligence -> LLM explanation -> FastAPI endpoints.

The service is vehicle-agnostic and does not require DBC decoding.

2) Why Isolation Forest

Isolation Forest is unsupervised and suitable for mixed, unlabeled CAN telemetry where normal and abnormal boundaries are unknown. It isolates rare behaviors using random partitioning and scales well for high-volume logs.

Random Forest is supervised; it requires labeled classes and is therefore not a fit for cold-start anomaly detection.

3) Feature Engineering

Core features: time_diff, message_frequency, window_frequency, byte_mean, byte_max, byte_std.

Advanced features: rolling_mean_8, rolling_std_8, payload_entropy.

These features capture timing behavior, bus activity intensity, and payload variability without OEM-specific decoding.

4) Improvements Implemented

- Parser expanded for candump, ASC Rx/Tx d lines, and generic ID#DATA fallback.
- Parse quality metrics added (total/parsed/skipped lines).
- Time-window frequency and rolling statistics added.
- Entropy feature added for payload variability.
- IsolationForest anomaly scoring converted to normalized severity (0..1).
- Automatic cluster count selection (elbow-style), plus cluster profiles.
- Model persistence added via joblib save/load.
- FastAPI hardened with request/response schemas and typed errors.
- Added /metrics endpoint for uptime, request counts, anomaly totals, and average latency.
- Added caching for repeated file analysis using SHA256 fingerprints.
- Added structured anomaly type classification: frequency/timing/data/pattern.
- LLM prompt upgraded to include severity, type, confidence, and cluster profile.

5) Limitations

- No DBC decoding by design. This preserves portability but reduces semantic signal naming precision.
- Statistical clusters are behavioral groups, not guaranteed OEM subsystem labels.
- LLM explanations depend on API availability and key configuration.

6) Example Output

```
{"can_id":"0x1A2","severity":0.92,"type":"frequency_anomaly","cluster":2,"confidence":"high"}
```

LLM output schema:

```
{"causes":"...","risks":"...","recommendations":"..."}
```

7) Real-world Relevance

This backend supports fleet-scale EV diagnostics, rapid anomaly triage, and explainable maintenance workflows.

It is designed for direct integration with external clients (including C# frontends) using

stable API contracts.